Alke Wallet

Evaluación del módulo 5

Fundamentos de bases de datos relacionales

Entrega: diseño conceptual, esquema relacional, implementación SQL y evidencias de ejecución.

Motor verificado: MySQL 8.0.46. Fecha de verificación: 2026-10-13T23:07:05.575382+00:00. Resultado: 31 pruebas aprobadas.

1. Objetivo y alcance

Se diseñó e implementó la base AlkeWallet para gestionar usuarios, saldos, monedas y transferencias. La solución cumple las tres entidades exigidas: usuario, transaccion y moneda. Los datos personales y correos son ficticios.

Cada usuario dispone de una wallet con una moneda actual. Una transferencia debita al emisor, acredita al receptor y registra su moneda histórica en una sola transacción. No se realizan conversiones; el cambio de moneda de una wallet requiere saldo cero.

Se utilizó Docker con MySQL 8 como plataforma equivalente a SQLOnline, opción admitida por el enunciado. La ejecución real y sus resultados se conservaron en archivos de texto, JSON y capturas. El módulo corresponde a una base de datos, por lo que esta entrega no requiere una interfaz web de la wallet.

Correspondencia con los requerimientos

Requisito	Evidencia entregada
Crear y conectar AlkeWallet	CREATE DATABASE, USE, SHOW DATABASES; captura 01.
Usuario, Transacción y Moneda	SQL 01: atributos, PK, FK, tipos y restricciones.
Moneda elegida por usuario	SQL 04: JOIN usuario / moneda con filtro.
Todas las transacciones	SQL 04: historial con nombres y moneda; captura 02.
Transacciones de un usuario	SQL 04: envíos e historial de envíos/recepciones.
Modificar correo y eliminar fila	SQL 05: UPDATE confirmado; DELETE ejecutado y revertido.
Integridad, ACID y modelo 3FN	Modelo ER; transferir; pruebas de rechazo, concurrencia y reinicio.
Word con SQL y capturas	Este documento: anexos completos y capturas de resultados reales.

2. Modelo conceptual y cardinalidades

Usuario representa a la persona propietaria de una wallet; Moneda identifica la divisa; Transacción representa un movimiento de fondos entre dos usuarios. Las tres son entidades fuertes porque poseen una clave primaria propia. La dependencia de existencia de una transacción respecto de sus participantes no la convierte en una entidad débil: su identificación no requiere una clave del propietario.

Relación	Cardinalidad y opcionalidad
Moneda / Usuario	Una moneda: 0..N usuarios. Cada usuario: exactamente 1 moneda actual.
Usuario / Transacción (envía)	Un usuario: 0..N envíos. Cada transacción: exactamente 1 emisor.
Usuario / Transacción (recibe)	Un usuario: 0..N recepciones. Cada transacción: exactamente 1 receptor.
Moneda / Transacción	Una moneda: 0..N transacciones. Cada transacción: exactamente 1 moneda histórica.

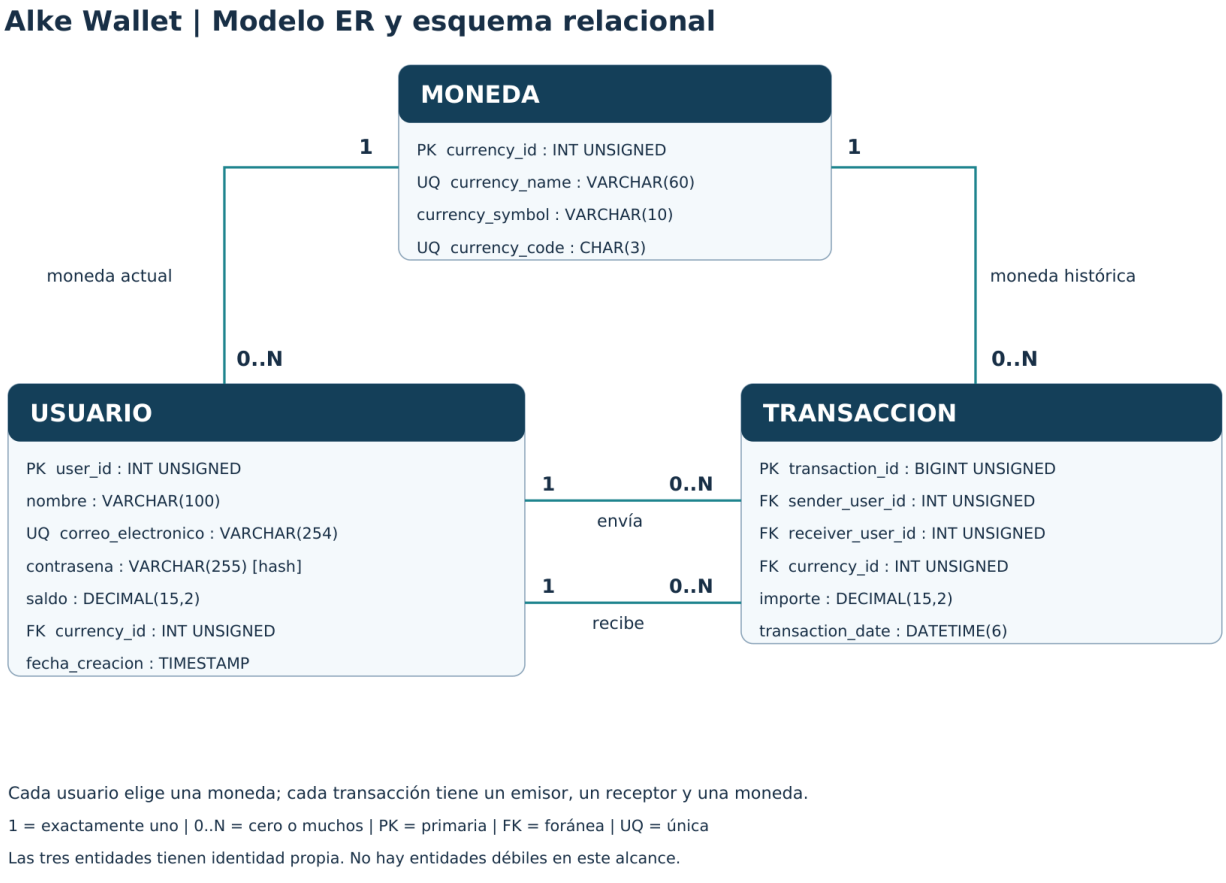


Figura 1. Modelo ER completo. Fuente editable: docs/modelo.dbml. Exportación PDF: output/pdf/modelo_entidad_relacion.pdf.

3. Transformación al modelo relacional y 3FN

Entidad	Clave y correspondencia relacional
Moneda	moneda(currency_id PK, currency_name UQ, currency_symbol, currency_code UQ).
Usuario	usuario(user_id PK, nombre, correo_electronico UQ, contrasena, saldo, currency_id FK, fecha_creacion).
Transacción	transaccion(transaction_id PK, sender_user_id FK, receiver_user_id FK, currency_id FK, importe, transaction_date).

Primera forma normal (1FN): cada celda almacena un valor atómico y cada fila tiene identificación propia. Se descartan listas de movimientos o monedas dentro de un usuario; cada movimiento ocupa una fila en transaccion.

Segunda forma normal (2FN): las claves primarias son simples; todos los atributos no clave dependen de su identificador completo, sin dependencias parciales.

Tercera forma normal (3FN): los nombres y símbolos de moneda se separan en moneda; los nombres y correos de los participantes se almacenan solo en usuario. Los JOIN recuperan las descripciones, evitando duplicación en el historial. currency_symbol no se declara único: distintas divisas pueden compartir un símbolo.

Dependencias funcionales: currency_id determina los atributos de moneda; user_id determina los atributos actuales del usuario; transaction_id determina los participantes, el importe, la fecha y la moneda del evento. currency_code, currency_name y correo_electronico son claves alternativas dentro de sus respectivas tablas.

La moneda histórica de una transacción no se deriva de la moneda actual del usuario: una wallet vacía puede elegir otra divisa conservando movimientos anteriores. Esta independencia fue probada. Así se evita la dependencia transitiva que surgiría si currency_id en transaccion solo repitiera una moneda permanentemente fija del emisor.

saldo es el estado actual de la wallet requerido por el enunciado; no se obtiene sumando solo transaccion, porque los saldos de apertura preceden al historial de transferencias. Los movimientos posteriores se aplican mediante transferir, que modifica ambos saldos y el historial conjuntamente.

Tipos, claves y reglas de integridad

Regla	Implementación
Precisión monetaria	DECIMAL(15,2) en saldos e importes; no se usa FLOAT.
Identidad e integridad referencial	PK autoincrementales, FK a usuarios y monedas, RESTRICT al borrar padres.
Datos obligatorios	NOT NULL en los atributos; nombre no vacío y formato básico de correo.
Coherencia de movimientos	CHECK saldo >= 0, importe > 0 y emisor distinto del receptor.
Unicidad	Correo de usuario, nombre y código de moneda mediante UNIQUE.
Moneda válida	Trigger al insertar un movimiento; cambio de moneda solo con saldo cero.
Búsquedas de historial	Índices compuestos por emisor/fecha y receptor/fecha.

4. Fundamentos y transaccionalidad

Una base relacional organiza información en tablas con filas y columnas. Las claves vinculan registros de distintas tablas, y las restricciones reducen datos duplicados, referencias inválidas y valores incoherentes. SQL permite definir estructuras y consultar o modificar datos de forma declarativa. Frente a archivos independientes, las relaciones y las transacciones facilitan mantener información consistente.

RDBMS	Disponibilidad y modelo
PostgreSQL	Software libre; servidor de bases de datos relacionales.
SQLite	Motor embebido de dominio público; apropiado para bases locales.
MySQL Community	Edición libre; arquitectura cliente / servidor. Motor usado en la entrega.
Oracle Database comercial	Licencia comercial; plataforma de administración empresarial.
Microsoft SQL Server comercial	Licencia propietaria; existen también ediciones gratuitas limitadas.

Ejemplo de tabla, fila y columna

La tabla usuario equivale a un registro organizado de personas. La fila de Ana Torres es un usuario; nombre, correo_electronico y saldo son columnas. user_id identifica a Ana, y currency_id conecta su wallet con la fila de Peso chileno en moneda.

Propiedades ACID

Propiedad	Aplicación y verificación
Atomicidad	Débito, crédito e INSERT se confirman juntos. Un handler ejecuta ROLLBACK y propaga cualquier error. Se probó una FK inválida después de un débito.
Consistencia	CHECK, UNIQUE, NOT NULL, FK y triggers validan reglas. El procedimiento exige saldo suficiente y monedas iguales.
Aislamiento	SELECT FOR UPDATE bloquea ambos usuarios por orden de ID. Dos gastos simultáneos de 50000 sobre 75000: solo uno confirma.
Durabilidad	InnoDB conserva COMMIT; los datos se verificaron desde otra sesión y tras reiniciar MySQL.

El procedimiento transferir inicia y finaliza su propia transacción: se invoca fuera de una transacción abierta. La carga inicial registra saldos de apertura y usa CALL para los movimientos. INSERT/UPDATE/DELETE directos se muestran para aprendizaje; los envíos de dinero se ejecutan con transferir para conservar la relación entre saldos e historial.

DELETE de una transacción se ejecuta dentro de una demostración reversible: se observa la fila eliminada y luego se restaura con ROLLBACK. No se confirma la eliminación de un movimiento financiero ya aplicado, porque eso dejaría el historial incompleto.

La vista adicional muestra hasta cinco usuarios por moneda. No ordena saldos de monedas diferentes como si fueran importes equivalentes. Se incluye además ALTER TABLE para fecha_creacion. La carga aleatoria de 50 transacciones es un ejercicio opcional del PDF y no forma parte de la instalación base.

5. Resultados y capturas reales

Las capturas siguientes se tomaron con un navegador sobre páginas que contienen la salida literal del cliente mysql. La ejecución corresponde a un servidor MySQL 8 en Docker, plataforma equivalente permitida por el enunciado. Los textos originales se adjuntan en output/evidencias; no se presentan como capturas de SQLOnline.

5.1. Creación de la base y conexión



Creación y conexión con AlkeWallet

Alke Wallet · MySQL 8.0.46 · Ejecución: 2026-10-13T23:07:05.575382+00:00

Salida literal del cliente mysql ejecutado en Docker. Esta página presenta resultados reales; no es una sesión de SQLOnline.

```
--------------
CREATE DATABASE AlkeWallet
    CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
--------------

--------------
SHOW DATABASES LIKE 'AlkeWallet'
--------------

+----------------------+
| Database (AlkeWallet) |
+----------------------+
| AlkeWallet           |
+----------------------+
--------------
SHOW DATABASES LIKE 'AlkeWallet'
--------------

+----------------------+
| Database (AlkeWallet) |
+----------------------+
| AlkeWallet           |
+----------------------+
--------------
SHOW TABLES
--------------

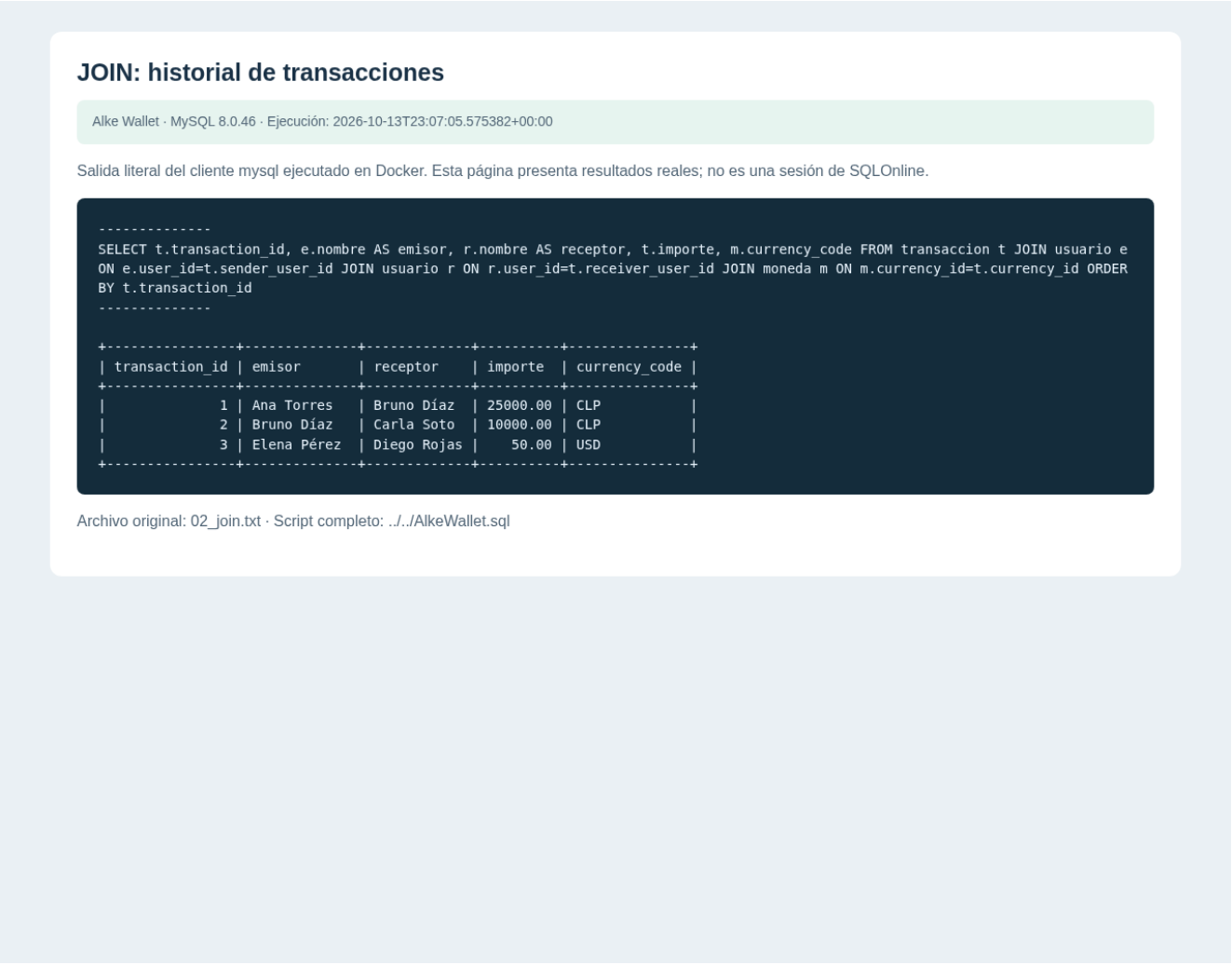
+---------------------+
| Tables_in_AlkeWallet |
+---------------------+
| moneda              |
| transaccion         |
| usuario             |
| vista_top_5_saldos  |
+---------------------+
--------------
SELECT VERSION() AS mysql_version, @@transaction_isolation AS aislamiento
--------------

+--------------+----------------+
| mysql_version | aislamiento    |
+--------------+----------------+
| 8.0.46       | REPEATABLE-READ |
+--------------+----------------+
```

Archivo original: 01_creacion.txt · Script completo: ../../AlkeWallet.sql

Captura 1. Archivo fuente: output/evidencias/01_creacion.txt.

5.2. Historial con JOIN



JOIN: historial de transacciones

Alke Wallet · MySQL 8.0.46 · Ejecución: 2026-10-13T23:07:05.575382+00:00

Salida literal del cliente mysql ejecutado en Docker. Esta página presenta resultados reales; no es una sesión de SQLOnline.

```
--------------
SELECT t.transaction_id, e.nombre AS emisor, r.nombre AS receptor, t.importe, m.currency_code FROM transaccion t JOIN usuario e
ON e.user_id=t.sender_user_id JOIN usuario r ON r.user_id=t.receiver_user_id JOIN moneda m ON m.currency_id=t.currency_id ORDER
BY t.transaction_id
--------------

+----------------+--------------+-------------+----------+---------------+
| transaction_id | emisor       | receptor    | importe  | currency_code |
+----------------+--------------+-------------+----------+---------------+
|              1 | Ana Torres   | Bruno Díaz  | 25000.00 | CLP           |
|              2 | Bruno Díaz   | Carla Soto  | 10000.00 | CLP           |
|              3 | Elena Pérez  | Diego Rojas |    50.00 | USD           |
+----------------+--------------+-------------+----------+---------------+
```

Archivo original: 02_join.txt · Script completo: ../../AlkeWallet.sql

Captura 2. Archivo fuente: output/evidencias/02_join.txt.

Ana envió 25000 CLP a Bruno; Bruno envió 10000 CLP a Carla; Elena envió 50 USD a Diego. El historial contiene tres transacciones confirmadas.

5.3. Estructura de las tres tablas

Tipos de datos, claves y nulidad

Alke Wallet · MySQL 8.0.46 · Ejecución: 2026-10-13T23:07:05.575382+00:00

Salida literal del cliente mysql ejecutado en Docker. Esta página presenta resultados reales; no es una sesión de SQLOnline.

```
-------------
DESCRIBE moneda
-------------

+-----------------+-------------+------+-----+---------+----------------+
| Field           | Type        | Null | Key | Default | Extra          |
+-----------------+-------------+------+-----+---------+----------------+
| currency_id     | int unsigned | NO   | PRI | NULL    | auto_increment |
| currency_name   | varchar(60)  | NO   | UNI | NULL    |                |
| currency_symbol | varchar(10)  | NO   |     | NULL    |                |
| currency_code   | char(3)      | NO   | UNI | NULL    |                |
+-----------------+-------------+------+-----+---------+----------------+
-------------
DESCRIBE usuario
-------------

+--------------------+---------------+------+-----+-------------------+-------------------+
| Field              | Type          | Null | Key | Default           | Extra             |
+--------------------+---------------+------+-----+-------------------+-------------------+
| user_id            | int unsigned  | NO   | PRI | NULL              | auto_increment    |
| nombre             | varchar(100)  | NO   |     | NULL              |                   |
| correo_electronico | varchar(254)  | NO   | UNI | NULL              |                   |
| contrasena         | varchar(255)  | NO   |     | NULL              |                   |
| saldo              | decimal(15,2) | NO   |     | 0.00              |                   |
| currency_id        | int unsigned  | NO   | MUL | NULL              |                   |
| fecha_creacion     | timestamp     | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+--------------------+---------------+------+-----+-------------------+-------------------+
-------------
DESCRIBE transaccion
-------------

+------------------+-----------------+------+-----+---------------------+-------------------+
| Field            | Type            | Null | Key | Default             | Extra             |
+------------------+-----------------+------+-----+---------------------+-------------------+
| transaction_id   | bigint unsigned | NO   | PRI | NULL                | auto_increment    |
| sender_user_id   | int unsigned    | NO   | MUL | NULL                |                   |
| receiver_user_id | int unsigned    | NO   | MUL | NULL                |                   |
| currency_id      | int unsigned    | NO   | MUL | NULL                |                   |
| importe          | decimal(15,2)   | NO   |     | NULL                |                   |
| transaction_date | datetime(6)     | NO   |     | CURRENT_TIMESTAMP(6) | DEFAULT_GENERATED |
+------------------+-----------------+------+-----+---------------------+-------------------+
```

Archivo original: 03_ddl.txt · Script completo: ../../AlkeWallet.sql

Captura 3. Archivo fuente: output/evidencias/03_ddl.txt.

5.4. COMMIT y ROLLBACK



COMMIT, error referencial y ROLLBACK

Alke Wallet · MySQL 8.0.46 · Ejecución: 2026-10-13T23:07:05.575382+00:00

Salida literal del cliente mysql ejecutado en Docker. Esta página presenta resultados reales; no es una sesión de SQLOnline.

```
--------------
START TRANSACTION
--------------

--------------
UPDATE usuario SET correo_electronico='ana.torres@example.com' WHERE user_id=1
--------------

--------------
COMMIT
--------------

--------------
SELECT user_id, saldo, correo_electronico FROM usuario WHERE user_id=1
--------------

+---------+----------+-----------------------+
| user_id | saldo    | correo_electronico     |
+---------+----------+-----------------------+
|       1 | 75000.00 | ana.torres@example.com |
+---------+----------+-----------------------+
--------------
CALL demostrar_error_referencial()
--------------

+----------------------------------------+
| resultado                              |
+----------------------------------------+
| Error de FK controlado: ROLLBACK completo |
+----------------------------------------+
--------------
START TRANSACTION
--------------

--------------
DELETE FROM transaccion WHERE transaction_id=1
--------------

--------------
SELECT COUNT(*) AS filas_tras_delete FROM transaccion WHERE transaction_id=1
--------------

+------------------+
| filas_tras_delete |
+------------------+
|                0 |
+------------------+
--------------
ROLLBACK
--------------

--------------
SELECT COUNT(*) AS filas_tras_rollback FROM transaccion WHERE transaction_id=1
--------------

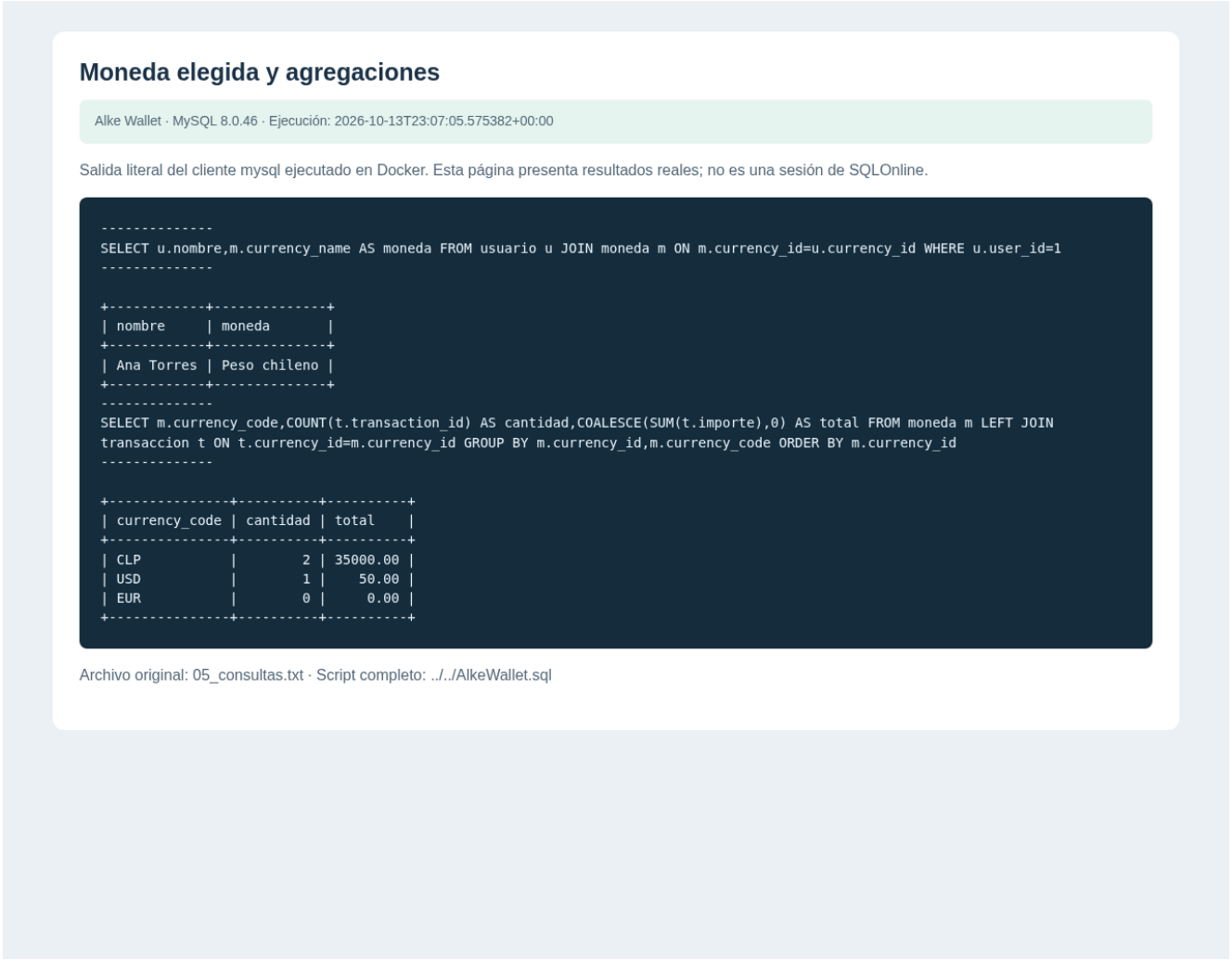
+--------------------+
| filas_tras_rollback |
+--------------------+
|                  1 |
+--------------------+
```

Archivo original: 04_commit_rollback.txt · Script completo: ../../AlkeWallet.sql

Captura 4. Archivo fuente: output/evidencias/04_commit_rollback.txt.

El correo de Ana quedó confirmado. El error referencial revierte el débito previo. DELETE muestra cero filas y ROLLBACK vuelve a mostrar una fila.

5.5. Moneda elegida y agregaciones



Moneda elegida y agregaciones

Alke Wallet · MySQL 8.0.46 · Ejecución: 2026-10-13T23:07:05.575382+00:00

Salida literal del cliente mysql ejecutado en Docker. Esta página presenta resultados reales; no es una sesión de SQLOnline.

```
--------------
SELECT u.nombre,m.currency_name AS moneda FROM usuario u JOIN moneda m ON m.currency_id=u.currency_id WHERE u.user_id=1
--------------

+------------+--------------+
| nombre     | moneda       |
+------------+--------------+
| Ana Torres | Peso chileno |
+------------+--------------+
--------------
SELECT m.currency_code,COUNT(t.transaction_id) AS cantidad,COALESCE(SUM(t.importe),0) AS total FROM moneda m LEFT JOIN
transaccion t ON t.currency_id=m.currency_id GROUP BY m.currency_id,m.currency_code ORDER BY m.currency_id
--------------

+---------------+----------+----------+
| currency_code | cantidad | total    |
+---------------+----------+----------+
| CLP           |        2 | 35000.00 |
| USD           |        1 |    50.00 |
| EUR           |        0 |     0.00 |
+---------------+----------+----------+
```

Archivo original: 05_consultas.txt · Script completo: ../../AlkeWallet.sql

Captura 5. Archivo fuente: output/evidencias/05_consultas.txt.

6. Validación y estado final

Se ejecutaron 31 pruebas sobre MySQL 8.0.46; todas finalizaron con estado OK. La prueba concurrente y la prueba de cambio de moneda restauran los datos de muestra al finalizar. Los IDs autoincrementales pueden presentar saltos después de operaciones rechazadas; no representan pérdida de registros.

#	Prueba	Resultado
1	Ejecución completa del SQL sin errores	OK
2	Tres tablas base	OK
3	Tres monedas, seis usuarios y tres transacciones	OK
4	Saldos iniciales y transferencias confirmadas	OK
5	Correo actualizado mediante COMMIT	OK
6	Moneda de un usuario	OK
7	Historial de envíos y recepciones	OK
8	Conservación de fondos por moneda	OK
9	Rechazo de correo duplicado	OK
10	Rechazo de saldo negativo	OK
11	Rechazo de importe cero en tabla	OK
12	Rechazo de auto transferencia en tabla	OK
13	FK rechaza receptor inexistente	OK
14	Trigger rechaza monedas incompatibles	OK
15	No se puede borrar un usuario con historial	OK
16	No se puede cambiar moneda con saldo disponible	OK
17	Saldo insuficiente revierte operación	OK
18	Rechazo de transferencias entre monedas	OK
19	Rechazo de auto transferencia en procedimiento	OK
20	Rechazo de importe negativo	OK
21	Rechazo de importe nulo	OK
22	Rechazo de participante nulo	OK
23	Rechazo de usuario inexistente	OK
24	Error de FK tras débito revierte saldo e historial	OK
25	DELETE elimina la fila y ROLLBACK la restaura	OK
26	Concurrencia: solo un gasto confirma y el otro falla	OK

#	Prueba	Resultado
27	Concurrencia: saldo correcto sin sobregiro	OK
28	Restauración de datos académicos tras prueba concurrente	OK
29	Moneda histórica independiente de la moneda actual	OK
30	Restauración tras cambio de moneda con saldo cero	OK
31	Durabilidad: COMMIT persiste tras reinicio de MySQL	OK

Saldos finales de la entrega

Usuario	Saldo	Moneda
Ana Torres	75000.00	CLP
Bruno Díaz	65000.00	CLP
Carla Soto	85000.00	CLP
Diego Rojas	350.00	USD
Elena Pérez	400.00	USD
Francisca León	200.00	EUR

Fondos conservados por divisa: 225000 CLP, 750 USD y 200 EUR. Tres movimientos iniciales, seis usuarios y tres monedas.

7. Reproducción de la entrega

Requisitos: Docker con Compose y Python 3. El contenedor es exclusivo de este proyecto y no publica puertos al host. No modifica bases de datos existentes de otros proyectos.

```
docker compose up -d --wait
python3 scripts/verificar.py
```

La verificación instala la base, ejecuta consultas, prueba integridad/concurrencia/durabilidad y escribe los resultados. Debe iniciarse con un contenedor nuevo: si AlkeWallet ya existe, el script se detiene para evitar sobrescribirla.

Para repetir desde cero exclusivamente la base de práctica de este proyecto: docker compose down --volumes; luego volver a levantar el servicio y ejecutar verificar.py. Este comando elimina el contenedor y su volumen anónimo de datos de demostración. Para conservar los datos basta usar docker compose stop y docker compose start.

Para instalar únicamente las sentencias, en un contenedor nuevo, puede usarse el cliente mysql:

```
docker compose exec -T mysql mysql -uroot < AlkeWallet.sql
```

README.md contiene el orden de archivos, instrucciones para SQLTools y reproducción de las pruebas SQL. docs/modelo.dbml permite editar el modelo con dbdiagram.io. El repositorio reúne el Word, la copia PDF, los SQL, el diagrama y las evidencias.

Fuentes

Enunciado de referencia: Proyecto Módulo #5 / ABP, Fundamentos de Bases de Datos Relacionales, páginas 2-8, utilizado para definir los requisitos de esta entrega.

MySQL 8.0 Reference Manual - CREATE TABLE:
https://dev.mysql.com/doc/refman/8.0/en/create-table.html

MySQL 8.0 Reference Manual - InnoDB Locks Set:
https://dev.mysql.com/doc/refman/8.0/en/innodb-locks-set.html

MySQL - CHECK constraints desde 8.0.16:
https://dev.mysql.com/blog-archive/mysql-8-0-16-introducing-check-constraint/

Anexo A. Sentencias SQL de la implementación

Se incluyen todas las sentencias de creación, carga, consulta, modificación, eliminación y transaccionalidad. Los archivos .sql adjuntos conservan el código ejecutable original; los saltos visuales del documento sirven para su lectura.

01_estructura.sql

```
-- Lección 1 y 4: creación de la base y de las tres entidades.
-- Ejecutar una sola vez en una base nueva; no se borran bases existentes.
CREATE DATABASE AlkeWallet
    CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE AlkeWallet;
SHOW DATABASES LIKE 'AlkeWallet';

CREATE TABLE moneda (
    currency_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    currency_name VARCHAR(60) NOT NULL,
    currency_symbol VARCHAR(10) NOT NULL,
    currency_code CHAR(3) NOT NULL,
    CONSTRAINT uq_moneda_nombre UNIQUE (currency_name),
    CONSTRAINT uq_moneda_codigo UNIQUE (currency_code)
) ENGINE = InnoDB;

CREATE TABLE usuario (
    user_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo_electronico VARCHAR(254) NOT NULL,
    contrasena VARCHAR(255) NOT NULL COMMENT 'Hash; nunca contraseña en texto plano',
    saldo DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    currency_id INT UNSIGNED NOT NULL,
    CONSTRAINT uq_usuario_correo UNIQUE (correo_electronico),
    CONSTRAINT chk_usuario_saldo CHECK (saldo >= 0),
    CONSTRAINT chk_usuario_nombre CHECK (CHAR_LENGTH(TRIM(nombre)) > 0),
    CONSTRAINT chk_usuario_correo CHECK (correo_electronico LIKE '%_@_%._%'),
    CONSTRAINT fk_usuario_moneda FOREIGN KEY (currency_id)
        REFERENCES moneda(currency_id) ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;

-- Plus de la lección 4: modificación de una tabla mediante ALTER TABLE.
ALTER TABLE usuario ADD COLUMN fecha_creacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP;

CREATE TABLE transaccion (
    transaction_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    sender_user_id INT UNSIGNED NOT NULL,
    receiver_user_id INT UNSIGNED NOT NULL,
    currency_id INT UNSIGNED NOT NULL,
    importe DECIMAL(15,2) NOT NULL,
    transaction_date DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    CONSTRAINT chk_transaccion_importe CHECK (importe > 0),
    CONSTRAINT chk_transaccion_participantes CHECK (sender_user_id <> receiver_user_id),
    CONSTRAINT fk_transaccion_emisor FOREIGN KEY (sender_user_id)
        REFERENCES usuario(user_id) ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_transaccion_receptor FOREIGN KEY (receiver_user_id)
        REFERENCES usuario(user_id) ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_transaccion_moneda FOREIGN KEY (currency_id)
        REFERENCES moneda(currency_id) ON DELETE RESTRICT ON UPDATE RESTRICT,
    INDEX idx_transaccion_emisor_fecha (sender_user_id, transaction_date),
    INDEX idx_transaccion_receptor_fecha (receiver_user_id, transaction_date)
) ENGINE = InnoDB;

-- Plus de lección 2: se comparan saldos dentro de la misma moneda.
CREATE VIEW vista_top_5_saldos AS
SELECT user_id, nombre, saldo, currency_code
FROM (
    SELECT u.user_id, u.nombre, u.saldo, m.currency_code,
           ROW_NUMBER() OVER (PARTITION BY u.currency_id ORDER BY u.saldo DESC, u.user_id) AS posicion
    FROM usuario u INNER JOIN moneda m ON m.currency_id = u.currency_id
) AS ranking
WHERE posicion <= 5;
```

02_transferencias.sql

```
USE AlkeWallet;
-- La moneda actual del usuario y la histórica de una transacción son atributos distintos.
-- El cambio de moneda solo es válido con saldo cero; no hay conversión de divisas.
DELIMITER $$
CREATE TRIGGER validar_cambio_moneda BEFORE UPDATE ON usuario
FOR EACH ROW
BEGIN
    IF NEW.currency_id <> OLD.currency_id AND (OLD.saldo <> 0 OR NEW.saldo <> 0) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El cambio de moneda requiere saldo cero';
    END IF;
END$$

CREATE TRIGGER validar_moneda_transaccion BEFORE INSERT ON transaccion
FOR EACH ROW
BEGIN
    DECLARE v_emisor INT UNSIGNED;
    DECLARE v_receptor INT UNSIGNED;
    SET v_emisor = (SELECT currency_id FROM usuario WHERE user_id = NEW.sender_user_id);
    SET v_receptor = (SELECT currency_id FROM usuario WHERE user_id = NEW.receiver_user_id);
    -- Si falta un participante, se deja a la FK generar su error de integridad.
    IF v_emisor IS NOT NULL AND v_receptor IS NOT NULL
       AND (NEW.currency_id <> v_emisor OR NEW.currency_id <> v_receptor) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Moneda de transaccion incompatible';
    END IF;
END$$
DELIMITER ;

-- Llamar fuera de una transacción abierta: el procedimiento administra COMMIT/ROLLBACK.
-- Orden fijo de bloqueo de usuarios para reducir conflictos concurrentes.
DELIMITER $$
CREATE PROCEDURE transferir(
    IN p_emisor INT UNSIGNED,
    IN p_receptor INT UNSIGNED,
    IN p_importe DECIMAL(15,2)
)
BEGIN
    DECLARE v_saldo DECIMAL(15,2);
    DECLARE v_moneda_emisor INT UNSIGNED;
    DECLARE v_moneda_receptor INT UNSIGNED;
    DECLARE v_id_bloqueado INT UNSIGNED;
    DECLARE v_falta_usuario BOOLEAN DEFAULT FALSE;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_falta_usuario = TRUE;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    IF p_emisor IS NULL OR p_receptor IS NULL OR p_emisor = p_receptor THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Participantes invalidos';
    END IF;
    IF p_importe IS NULL OR p_importe <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El importe debe ser positivo';
    END IF;

    START TRANSACTION;
    SELECT user_id INTO v_id_bloqueado FROM usuario
        WHERE user_id = LEAST(p_emisor, p_receptor) FOR UPDATE;
    SELECT user_id INTO v_id_bloqueado FROM usuario
        WHERE user_id = GREATEST(p_emisor, p_receptor) FOR UPDATE;
    IF v_falta_usuario THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Usuario inexistente';
    END IF;
    SELECT saldo, currency_id INTO v_saldo, v_moneda_emisor
        FROM usuario WHERE user_id = p_emisor;
    SELECT currency_id INTO v_moneda_receptor
        FROM usuario WHERE user_id = p_receptor;
    IF v_moneda_emisor <> v_moneda_receptor THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Los usuarios deben utilizar la misma moneda';
    END IF;
    IF v_saldo < p_importe THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Saldo insuficiente';
    END IF;

    UPDATE usuario SET saldo = saldo - p_importe WHERE user_id = p_emisor;
    UPDATE usuario SET saldo = saldo + p_importe WHERE user_id = p_receptor;
    INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)
        VALUES (p_emisor, p_receptor, v_moneda_emisor, p_importe);
    COMMIT;
END$$
DELIMITER ;
```

03_datos.sql

```
USE AlkeWallet;
-- Datos ficticios. El hash bcrypt es un valor de prueba, sin autenticación en este módulo.
START TRANSACTION;
INSERT INTO moneda (currency_id, currency_name, currency_symbol, currency_code) VALUES
    (1, 'Peso chileno', '$', 'CLP'),
    (2, 'Dólar estadounidense', 'US$', 'USD'),
    (3, 'Euro', '€', 'EUR');
INSERT INTO usuario (user_id, nombre, correo_electronico, contrasena, saldo, currency_id) VALUES
    (1, 'Ana Torres', 'ana@example.com', '$2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW',
  100000.00, 1),
    (2, 'Bruno Díaz', 'bruno@example.com', '$2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW',
  50000.00, 1),
    (3, 'Carla Soto', 'carla@example.com', '$2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW',
  75000.00, 1),
    (4, 'Diego Rojas', 'diego@example.com', '$2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW',
  300.00, 2),
    (5, 'Elena Pérez', 'elena@example.com', '$2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW',
  450.00, 2),
    (6, 'Francisca León', 'francisca@example.com',
  '$2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW', 200.00, 3);
COMMIT;

-- Las tres transacciones iniciales modifican los saldos y registran el historial juntas.
CALL transferir(1, 2, 25000.00);
CALL transferir(2, 3, 10000.00);
CALL transferir(5, 4, 50.00);
```

04_consultas.sql

```
USE AlkeWallet;
SET @usuario_id = 1;

-- Requerimiento: nombre de la moneda elegida por un usuario específico.
SELECT u.user_id, u.nombre, m.currency_name AS moneda, m.currency_symbol, m.currency_code
FROM usuario u INNER JOIN moneda m ON m.currency_id = u.currency_id
WHERE u.user_id = @usuario_id;

-- Requerimiento: todas las transacciones (JOIN exitoso con sus participantes).
SELECT t.transaction_id, emisor.nombre AS emisor, receptor.nombre AS receptor,
       t.importe, m.currency_code, t.transaction_date
FROM transaccion t
INNER JOIN usuario emisor ON emisor.user_id = t.sender_user_id
INNER JOIN usuario receptor ON receptor.user_id = t.receiver_user_id
INNER JOIN moneda m ON m.currency_id = t.currency_id
ORDER BY t.transaction_id;

-- Requerimiento: todas las transacciones realizadas por un usuario (enviadas).
SELECT * FROM transaccion WHERE sender_user_id = @usuario_id ORDER BY transaction_date, transaction_id;

-- Historial completo: tanto envíos como recepciones.
SELECT transaction_id, sender_user_id, receiver_user_id, importe, currency_id, transaction_date
FROM transaccion
WHERE sender_user_id = @usuario_id OR receiver_user_id = @usuario_id
ORDER BY transaction_date, transaction_id;

-- SELECT básico y filtros parametrizados con operadores lógicos.
SELECT user_id, nombre, correo_electronico, saldo, currency_id FROM usuario ORDER BY user_id;
SET @moneda_id = 1;
SET @saldo_minimo = 60000.00;
SELECT user_id, nombre, saldo FROM usuario
WHERE currency_id = @moneda_id AND saldo >= @saldo_minimo ORDER BY saldo DESC;

-- Subconsulta correlacionada: incluye usuarios sin transacciones.
SELECT u.user_id, u.nombre,
       (SELECT COUNT(*) FROM transaccion t
        WHERE t.sender_user_id = u.user_id OR t.receiver_user_id = u.user_id) AS total_transacciones
FROM usuario u ORDER BY u.user_id;

-- COUNT y SUM con GROUP BY: no se suman monedas diferentes.
SELECT m.currency_code, COUNT(t.transaction_id) AS cantidad, COALESCE(SUM(t.importe), 0) AS
  total_transferido
FROM moneda m LEFT JOIN transaccion t ON t.currency_id = m.currency_id
GROUP BY m.currency_id, m.currency_code ORDER BY m.currency_id;
SELECT * FROM vista_top_5_saldos ORDER BY currency_code, saldo DESC, user_id;
```

05_dml_y_rollback.sql

```
USE AlkeWallet;
-- Requerimiento: modificar el correo de un usuario específico y confirmar.
START TRANSACTION;
UPDATE usuario SET correo_electronico = 'ana.torres@example.com' WHERE user_id = 1;
COMMIT;
SELECT user_id, nombre, correo_electronico FROM usuario WHERE user_id = 1;

-- ROLLBACK explícito de una modificación de saldo: el valor final se conserva.
SELECT saldo INTO @saldo_antes FROM usuario WHERE user_id = 1;
START TRANSACTION;
UPDATE usuario SET saldo = saldo - 100.00 WHERE user_id = 1;
SELECT saldo AS saldo_temporal FROM usuario WHERE user_id = 1;
ROLLBACK;
SELECT saldo AS saldo_final, @saldo_antes AS saldo_original FROM usuario WHERE user_id = 1;

-- Requerimiento: eliminar una fila completa de transaccion.
-- Ejemplo académico reversible para conservar el historial y los saldos de la entrega.
START TRANSACTION;
DELETE FROM transaccion WHERE transaction_id = 1;
SELECT COUNT(*) AS filas_despues_de_delete FROM transaccion WHERE transaction_id = 1;
ROLLBACK;
SELECT COUNT(*) AS filas_despues_de_rollback FROM transaccion WHERE transaction_id = 1;
```

06_error_referencial.sql

```
USE AlkeWallet;
-- El procedimiento demuestra un error de FK después de modificar el saldo.
-- Un handler revierte TODA la operación, no solo el INSERT fallido.
DELIMITER $$
CREATE PROCEDURE demostrar_error_referencial()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error de FK controlado: ROLLBACK completo' AS resultado;
    END;
    START TRANSACTION;
    UPDATE usuario SET saldo = saldo - 100.00 WHERE user_id = 1;
    INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)
        VALUES (1, 999999, 1, 100.00);
    COMMIT;
END$$
DELIMITER ;
CALL demostrar_error_referencial();
SELECT user_id, saldo FROM usuario WHERE user_id = 1;
```

07_inspeccion.sql

```
USE AlkeWallet;
SHOW TABLES;
DESCRIBE moneda;
DESCRIBE usuario;
DESCRIBE transaccion;
SHOW CREATE TABLE moneda;
SHOW CREATE TABLE usuario;
SHOW CREATE TABLE transaccion;
SELECT VERSION() AS version_mysql, @@transaction_isolation AS aislamiento,
       @@innodb_flush_log_at_trx_commit AS persistencia_commit;
SELECT m.currency_code, SUM(u.saldo) AS saldo_total
FROM usuario u INNER JOIN moneda m ON m.currency_id = u.currency_id
GROUP BY m.currency_id, m.currency_code ORDER BY m.currency_id;
```

Anexo B. Sentencias adicionales de validación

Registro sin repeticiones de las sentencias ejecutadas por el verificador. Contiene errores intencionales, comprobaciones y restauración de pruebas: no se utiliza como script de instalación. La concurrencia la administra scripts/verificar.py mediante dos sesiones simultáneas.

```
-- Sentencias efectivamente ejecutadas por verificar.py, sin repeticiones.
-- Archivo de evidencia: incluye errores intencionales y pruebas concurrentes.
-- No se ejecuta como script de instalación.

SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='AlkeWallet' AND table_type='BASE TABLE';

SELECT (SELECT COUNT(*) FROM moneda), (SELECT COUNT(*) FROM usuario), (SELECT COUNT(*) FROM transaccion);

SELECT saldo FROM usuario ORDER BY user_id;

SELECT correo_electronico FROM usuario WHERE user_id=1;

SELECT m.currency_name FROM usuario u JOIN moneda m ON m.currency_id=u.currency_id WHERE u.user_id=1;

SELECT COUNT(*) FROM transaccion WHERE sender_user_id=2 OR receiver_user_id=2;

SELECT currency_id, SUM(saldo) FROM usuario GROUP BY currency_id ORDER BY currency_id;

SELECT user_id, saldo, correo_electronico, currency_id FROM usuario ORDER BY user_id;

SELECT transaction_id, sender_user_id, receiver_user_id, currency_id, importe FROM transaccion ORDER BY
  transaction_id;

UPDATE usuario SET correo_electronico='bruno@example.com' WHERE user_id=1;

UPDATE usuario SET saldo=-1 WHERE user_id=1;

INSERT INTO transaccion(sender_user_id,receiver_user_id,currency_id,importe) VALUES(1,2,1,0);

INSERT INTO transaccion(sender_user_id,receiver_user_id,currency_id,importe) VALUES(1,1,1,1);

INSERT INTO transaccion(sender_user_id,receiver_user_id,currency_id,importe) VALUES(1,999999,1,1);

INSERT INTO transaccion(sender_user_id,receiver_user_id,currency_id,importe) VALUES(1,4,1,1);

DELETE FROM usuario WHERE user_id=1;

UPDATE usuario SET currency_id=2 WHERE user_id=1;

CALL transferir(1,2,999999.00);

CALL transferir(1,4,1.00);

CALL transferir(1,1,1.00);

CALL transferir(1,2,-1.00);

CALL transferir(1,2,NULL);

CALL transferir(NULL,2,1.00);

CALL transferir(1,999999,1.00);

CALL demostrar_error_referencial();

START TRANSACTION; DELETE FROM transaccion WHERE transaction_id=1; SELECT COUNT(*) FROM transaccion WHERE
  transaction_id=1; ROLLBACK;

CALL transferir(1,2,50000.00);

CALL transferir(1,3,50000.00);

SELECT saldo FROM usuario WHERE user_id=1;

SELECT receiver_user_id FROM transaccion ORDER BY transaction_id DESC LIMIT 1;

CALL transferir(2,1,50000.00);

START TRANSACTION; DELETE FROM transaccion WHERE transaction_id > 3; COMMIT;

CALL transferir(1,2,75000.00);

SELECT u.currency_id, t.currency_id FROM usuario u JOIN transaccion t ON t.sender_user_id=u.user_id WHERE
  u.user_id=1 AND t.transaction_id=1;

UPDATE usuario SET currency_id=1 WHERE user_id=1;

CALL transferir(2,1,75000.00);
```

```
SHOW DATABASES LIKE 'AlkeWallet'; SHOW TABLES; SELECT VERSION() AS mysql_version, @@transaction_isolation AS
   aislamiento;

SELECT t.transaction_id, e.nombre AS emisor, r.nombre AS receptor, t.importe, m.currency_code FROM
  transaccion t JOIN usuario e ON e.user_id=t.sender_user_id JOIN usuario r ON r.user_id=t.receiver_user_id
  JOIN moneda m ON m.currency_id=t.currency_id ORDER BY t.transaction_id;

DESCRIBE moneda; DESCRIBE usuario; DESCRIBE transaccion;

START TRANSACTION; UPDATE usuario SET correo_electronico='ana.torres@example.com' WHERE user_id=1; COMMIT;
  SELECT user_id, saldo, correo_electronico FROM usuario WHERE user_id=1; CALL
  demostrar_error_referencial(); START TRANSACTION; DELETE FROM transaccion WHERE transaction_id=1; SELECT
  COUNT(*) AS filas_tras_delete FROM transaccion WHERE transaction_id=1; ROLLBACK; SELECT COUNT(*) AS
  filas_tras_rollback FROM transaccion WHERE transaction_id=1;

SELECT u.nombre,m.currency_name AS moneda FROM usuario u JOIN moneda m ON m.currency_id=u.currency_id WHERE
  u.user_id=1; SELECT m.currency_code,COUNT(t.transaction_id) AS cantidad,COALESCE(SUM(t.importe),0) AS
  total FROM moneda m LEFT JOIN transaccion t ON t.currency_id=m.currency_id GROUP BY
  m.currency_id,m.currency_code ORDER BY m.currency_id;

SELECT VERSION();

SELECT user_id,nombre,saldo,currency_id FROM usuario ORDER BY user_id;

SELECT transaction_id,sender_user_id,receiver_user_id,importe FROM transaccion ORDER BY transaction_id;
```